



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE APPLICATION: CAPITAL ASSET PRICING MODEL - SOLUTION

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



Developed by William Sharpe (1964), the CAPM (*Capital Asset Pricing Model*) starts from the idea that every investment carries two types of risk: systematic (market risk that cannot be diversified away) and unsystematic (the component of a stock's return that is not correlated with market movements). portfolio diversification can remove unsystematic risk; however, this is not the case for systematic risk.

The CAPM is a way of measuring systematic risk. William Sharpe finds that the return of a stock or portfolio is:

$$R_{i,t} = R_f + \beta_i(R_{m,t} - R_f)$$

Where:

- $R_{i,t}$: Expected return of the portfolio or stock.
- R_f : Risk-free rate.
- $R_{m,t}$: Expected market return.
- $R_{m,t} - R_f$: *Equity market premium*
- β_i : The CAPM beta.

According to the CAPM, β is the only relevant measure of risk, defined as:

$$\beta_i = \frac{\text{cov}(R_{p,t}, R_{m,t})}{\sigma_m^2}$$

If β is taken as the second coefficient of a simple OLS linear regression:

$$R_{i,t} - R_f = \alpha_i + \beta_i(R_{m,t} - R_f) + \varepsilon_{i,t}$$

Although the model above is not the CAPM, the regression allows the CAPM β to be estimated. In fact, if $\alpha_i = 0$ and $\text{Cov}(R_{m,t}, \varepsilon_{i,t}) = 0$ then:

$$\begin{aligned} R_{i,t} - R_f &= \beta_i(R_{m,t} - R_f) \\ R_{i,t} &= R_f + \beta_i(R_{m,t} - R_f) \end{aligned}$$

The CAPM equation is obtained.

1 Application

1. Import the files `stocks.pkl`, `sp500.pkl` and `rfree.pkl`.

- `stocks.pkl`: indices for Facebook, Apple, Amazon, Netflix and Google from 2014:12 to 2020:12.
- `sp500.pkl`: Closing price of the S&P 500 index from 2014:12 to 2020:12.
- `rfree.pkl`: Risk-free rate from 2015:01 to 2020:12.

```
import numpy as np
import pandas as pd

# load indices
stocks = pd.read_pickle('stocks.pkl')

# load sp 500
sp500 = pd.read_pickle('sp500.pkl')

# risk-free rate
Rf = pd.read_pickle('rfree.pkl')
```

2. Write a function that computes the arithmetic return, defined as:

$$R_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}} \quad i = 1, \dots, 5 \quad t = 1, \dots, 73$$

```
# create the function
def returns(x):

    '''
    Parameters
    -----
    x : Array of dimension tx1

    Returns
    -----
    Arithmetic return of dimension (t-1)x1
    '''

    arithmetic_return = np.diff(x, axis = 0) / x[:-1]
    return(arithmetic_return)
```

3. Apply the function created in (2) to each element (closing prices) stored in the dictionary.

```
# empty dictionary
dict_return = {}

for ticker, close in stocks.items():
    print('Computing return for ticker: ', ticker)
    dict_return[ticker] = returns(close)
```

```
## Computing return for ticker: AAPL
## Computing return for ticker: AMZN
## Computing return for ticker: FB
## Computing return for ticker: GOOG
## Computing return for ticker: NFLX
```

4. Apply the function created in (2) to the S&P 500 index. Then subtract the risk-free rate.

$$R_{m,t} = R_{sp500} - R_f$$

```
# market return
Rm = returns(sp500)

# risk premium
Rm_Rf = Rm - Rf
```

5. Write a function that weights each return by a fixed weight. The weights must sum to 1 and the *output* must have the risk-free rate R_f subtracted.

$$R_{p,t} = \mathbf{R} \cdot \mathbf{W} = \begin{pmatrix} r_{11} & r_{12} & \dots & r_{1i} \\ r_{21} & r_{22} & \dots & r_{2i} \\ \vdots & \vdots & \ddots & \vdots \\ r_{t1} & r_{t2} & \dots & r_{ti} \end{pmatrix}_{t \times i} \cdot \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_i \end{pmatrix}_{i \times 1}$$

Test your function with the following weights:

Table 1: Index & weights

Index	Weights
AAPL	0.25
AMZN	0.25
FB	0.20
GOOG	0.20
NFLX	0.10

```
# weights (sum to 1)
w = np.array([0.25, 0.25, 0.20, 0.20, 0.10]).reshape((-1,1))

# create the function
def portfolio(x, w, rf):

    '''
    Parameters
    -----
    x : Dictionary containing i arrays of dimension t1
    w : Weights of dimension i1
    rf: Risk-free rate of dimension t1

    Returns
    -----
    Weighted portfolio of dimension t1
    '''

    ret_list = []

    for name, r_i in x.items():
        ret_list.append(r_i)

    ret_mat = np.concatenate(ret_list, axis=1)
    rp = ret_mat.dot(w) - rf

    return(rp)

# excess return
Rp_Rf = portfolio(dict_return, w, Rf)
```

6. Write a function that obtains the $\hat{\beta}$ of a linear regression using its matrix form:

$$\hat{\beta} = (\mathbf{X}'\mathbf{X})^{-1}(\mathbf{X}'y)$$

```
# create the function
def ols(x,y):

    '''
    Parameters
    -----
    x : ti matrix (Rm - Rf)
    y : ti matrix (Rp - Rf)

    Returns
    -----
    Alpha and Beta
    '''

    m,n = x.shape

    x_with_ones = np.hstack((np.ones((m,1)),x))

    x_t_x = x_with_ones.T.dot(x_with_ones)
    inv_x_t_x = np.linalg.inv(x_t_x)

    betas = inv_x_t_x.dot(x_with_ones.T.dot(y))

    return(betas)

# parameters
ols(Rm_Rf,Rp_Rf)

## array([[0.0172864 ],
##        [1.12976075]])
```

7. Show that the previous result is equal to:

$$\hat{\beta} = \frac{cov(R_{p,t}, R_{m,t})}{\sigma_m^2}$$

```
# beta portfolio
cov_var_mat = np.cov(np.concatenate([Rp_Rf,Rm_Rf], axis = 1), rowvar=False)

print('The CAPM Beta is: ', cov_var_mat[0,1] / cov_var_mat[1,1])

## The CAPM Beta is:  1.1297607468830584
```

8. Verify the result obtained in (6) using the statsmodels library.

```
import statsmodels.api as sm

# add a column of ones
Rm_Rf_with_const = sm.add_constant(Rm_Rf)

# OLS estimation
mod = sm.OLS(Rp_Rf,Rm_Rf_with_const)
# model fit
res = mod.fit()
```

```
# result
print(res.summary(res))

res.params # parameters

## array([0.0172864 , 1.12976075])
```